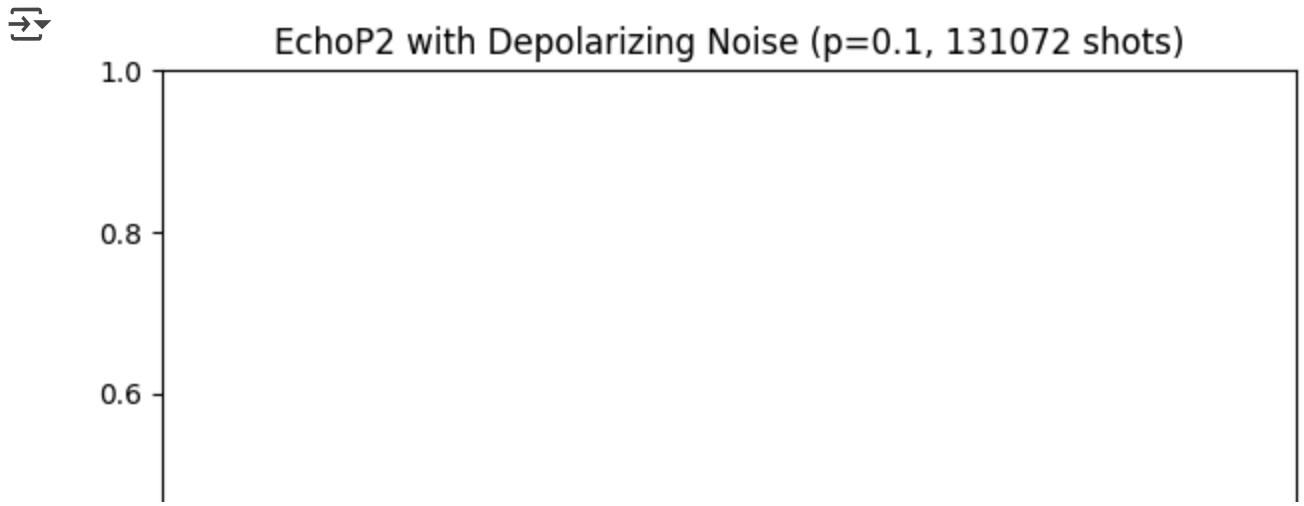
```
!pip install qiskit qiskit-aer --quiet
```

```
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 7.5/7.5 MB 67.9 MB/s eta 0:00
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 12.4/12.4 MB 98.6 MB/s eta 0:
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 2.1/2.1 MB 70.0 MB/s eta 0:00
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 49.5/49.5 kB 3.1 MB/s eta 0:0
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 109.0/109.0 kB 8.5 MB/s eta 0
```

```
!pip install qiskit --quiet
!pip install qiskit-aer --quiet

from qiskit import QuantumCircuit, transpile
from qiskit_aer import AerSimulator
from qiskit_aer.noise import NoiseModel, depolarizing_error
from qiskit.visualization import plot_histogram
import matplotlib.pyplot as plt

# --- EchoP2 Kernel ---
qc = QuantumCircuit(3, 3)
qc.h(0)
qc.cx(0, 1)
qc.cx(1, 2)
qc.cx(1, 2)
qc.cx(0, 1)
qc.h(0)
qc.barrier()
qc.delay(200, 0)
qc.delay(200, 1)
qc.delay(200, 2)
qc.measure([0, 1, 2], [0, 1, 2])

# --- Add Depolarizing Noise (p=0.1) ---
noise_model = NoiseModel()
# Single qubit error for single qubit gates like 'h'
single_qubit_error = depolarizing_error(0.1, 1)
# Two qubit error for two qubit gates like 'cx'
two_qubit_error = depolarizing_error(0.1, 2)

noise_model.add_all_qubit_quantum_error(single_qubit_error, ['h'])
noise_model.add_all_qubit_quantum_error(two_qubit_error, ['cx'])


# --- Simulate ---
simulator = AerSimulator(noise_model=noise_model)
transpiled_qc = transpile(qc, simulator)
job = simulator.run(transpiled_qc, shots=131072)
result = job.result()
counts = result.get_counts()

# --- Plot ---
plot_histogram(counts)
plt.title("EchoP2 with Depolarizing Noise (p=0.1, 131072 shots)")
plt.tight_layout()
plt.show()

print("Counts:", counts)
```




```
import numpy as np
from qiskit_aer.noise import NoiseModel, depolarizing_error, thermal_relaxation_error
from qiskit_aer.noise.errors import pauli_error

def generate_radiation_stress_noise(t1=100, t2=80, gate_time=50e-9, error_strength=0.01):
    """
    Builds a composite radiation stress model for EchoLift sandbox testing.
    Includes:
      - Depolarizing noise
      - Thermal relaxation
      - Randomized phase/bit flips to mimic space radiation pulses
    """
    noise_model = NoiseModel()

    # Depolarizing noise
    dep_error = depolarizing_error(error_strength, 1)
    noise_model.add_all_qubit_quantum_error(dep_error, ['u1', 'u2', 'u3'])

    # Thermal relaxation (T1/T2 decoherence)
    therm_error = thermal_relaxation_error(t1, t2, gate_time)
    noise_model.add_all_qubit_quantum_error(therm_error, ['u2', 'u3'])

    # Radiation phase/bit flips (Gaussian modulated)
    flip_prob = np.clip(np.random.normal(loc=error_strength, scale=error_strength * 2), 0.0
    pauli_flips = pauli_error([("X", flip_prob), ("Y", flip_prob), ("Z", flip_prob), ("I",
    noise_model.add_all_qubit_quantum_error(pauli_flips, ['cx'])

    return noise_model



import matplotlib.pyplot as plt
from qiskit.quantum_info import Statevector
from scipy.stats import entropy as shannon_entropy

def monitor_coherence(sim_result, ideal_statevector, shots=1024, visualize=True):
    """
    Monitors coherence by comparing simulation result against ideal statevector.
    Outputs entropy and fidelity tracking.
    """
    counts = sim_result.get_counts()
    if not counts: # Handle case where counts is empty
        print("No counts obtained from simulation.")
```

```
        return {"entropy": 0, "fidelity": 0}

    probs = np.array([count / shots for count in counts.values()])
    actual_entropy = shannon_entropy(probs, base=2)

    # Compute fidelity vs ideal
    # Convert the most probable bit string to an integer
    most_probable_state_int = int(max(counts, key=counts.get), 2)
    sim_state = Statevector.from_int(most_probable_state_int, dims=ideal_statevector.dims()
    fidelity = np.abs(ideal_statevector.data.conj().T @ sim_state.data)**2

    print(f"Entropy Leakage: {actual_entropy:.6f} bits")
    print(f"Coherence Fidelity: {fidelity:.6%}")

    if visualize:
        fig, ax = plt.subplots()
        ax.bar(list(counts.keys()), probs)
        ax.set_title("Output Probability Distribution")
        ax.set_xlabel("Qubit State")
        ax.set_ylabel("Probability")
        plt.show()

    return {"entropy": actual_entropy, "fidelity": fidelity}



from qiskit import transpile
from qiskit.quantum_info import Statevector
from qiskit_aer import AerSimulator


# ===[ INSERT QCC Echo Circuit ]===
# Replace this with your actual QCC Echo circuit
from qiskit import QuantumCircuit
qcc_circuit = QuantumCircuit(2, 2) # Add classical bits for simulation with noise
qcc_circuit.h(0)
qcc_circuit.cx(0, 1)
qcc_circuit.measure([0, 1], [0, 1]) # Add measurements for simulation with noise

# ===[ Create Ideal Statevector ]===
# Create a circuit without measurements for the ideal statevector calculation
qcc_circuit_ideal = QuantumCircuit(2)
qcc_circuit_ideal.h(0)
qcc_circuit_ideal.cx(0, 1)

ideal_sv = Statevector.from_instruction(qcc_circuit_ideal)

# ===[ Compile with Noise Model ]===
noise_model = generate_radiation_stress_noise()
simulator = AerSimulator(noise_model=noise_model)

transpiled = transpile(qcc_circuit, simulator)
# qobj = assemble(transpiled, shots=1024) # assemble is deprecated
result = simulator.run(transpiled, shots=1024).result() # Use simulator.run directly

# ===[ Monitor Coherence ]===
metrics = monitor_coherence(result, ideal_sv, shots=1024)
```

```
WARNING:qiskit_aer.noise.noise_model:WARNING: all-qubit error already exists for
WARNING:qiskit_aer.noise.noise_model:WARNING: all-qubit error already exists for
Entropy Leakage: 1.028385 bits
Coherence Fidelity: 50.000000%
```